Q&A Preparation

Questions the panel is likely to ask, grouped, with answers you can say out loud. Where a number is involved, the file it comes from is named so you can defend it.

Three rules for the room.

1. If you do not know, say "I don't know — that is in the part I did not build, but I can tell you where it is measured." Guessing is the only way to actually lose marks here.
2. Never oversell. Our claim is narrow and specific and it survives scrutiny: *signature rules break when you reword an attack; embeddings do not.* The broad claim ("more robust to camouflage") is **false** by our own measurements and they will find it.
3. When asked about a weakness, give the number, then the cause, then the fix.

A. The basics — expect these from anyone

Q. What does your project actually do? It reads system log files, flags lines that look unusual for the user who produced them, groups related ones into a single incident, labels the incident with the standard MITRE attack code, and has a small language model running on the same laptop write a plain-English explanation of what happened.

Q. What is an embedding, in one sentence? A way of turning a sentence into a list of numbers so that sentences meaning similar things get similar numbers — which lets the computer compare meaning instead of spelling.

Q. Why Isolation Forest and not a neural network? Because it needs no labelled attack data, and no real organisation has labelled logs. It also runs on a CPU in seconds. A 2025 journal study in Empirical Software Engineering (reference R3, in the attached PDFs) compared classical machine learning with deep learning on exactly this task and found the simple methods competitive.

Q. What is MITRE ATT&CK? A public catalogue of attacker techniques with codes — T1110 is Brute Force. Security teams talk in those codes, so attaching them makes our output usable by a real analyst.

Q. Where does the language model fit in? Only at the end, and only to write prose. It never decides whether something is an attack and it is never allowed to choose the MITRE code — those come from a lookup table. This is deliberate: if the model picked the codes it could invent one that does not exist.

Q. Does it need the internet? No. Detection, grouping, MITRE mapping and the written explanation all run locally. There is no cloud API and no API key anywhere in the system.

B. Method — why you built it this way

Q. Why not fine-tune a language model on logs, as your first proposal said? Three reasons. It does not fix the actual problem, which is precision, not understanding. It makes results much harder for anyone else to reproduce. And it needs a GPU, which would break the "runs on any laptop" property we consider a feature. We changed this deliberately and recorded why.

Q. Isn't Isolation Forest over embeddings a standard, well-known combination? Yes, and we say so on the methodology slide. We do not claim the detector as the novelty. The contribution is the measurement: nobody publishes how much detection is lost when the attacker is actively hiding, measured against a production SIEM on the same logs.

Q. Why one model per user instead of one model for everything? Because "unusual" is personal. A database administrator logging in at 3 a.m. is normal; a receptionist doing it is not. One global model would flatten that distinction.

Q. Why do you hide the IP address and port before embedding? So that two events of the same kind look the same. Otherwise the model would treat `Failed password from 10.0.0.5` and `Failed password from 10.0.0.9` as different events when they are the same event from different places.

Q. How do you catch an attack that spans several lines? By grouping: suspicious events from one user inside a 30-minute window become one incident. There is also a physics check for impossible travel — one account logging in from two places too far apart for the time between them.

Q. Which part catches impossible travel — the machine learning or the rule? The rule, and we are explicit about that. After masking the IP address, both login lines look identical to the embeddings, so they contribute nothing there. Saying otherwise would be a false claim, and it is in our own list of known issues.

C. Evaluation — the questions that decide your marks

Q. How do you know it works? Three ways. Line-level precision, recall and F1 against a corpus where we know every answer. Instance detection — did we notice the attack at all — on real logs with attacks injected. And robustness: how far detection falls when the attack is camouflaged, run against a real Wazuh SIEM over five random seeds.

Q. Why Wazuh and not Splunk or QRadar? Because we needed a real production rule engine we could run hundreds of times automatically. Wazuh is a genuine SIEM used in production, it is free, and we run the actual manager in Docker with its own rules — we did not reimplement it. Splunk and QRadar are commercial and cannot be scripted into a repeatable experiment.

Q. Did you write the rules you are comparing against? No, and that is the whole point. In our earlier version we wrote both the attacks and the rules that caught them, which made the comparison meaningless. Wazuh's rules were written by people who have never seen our data.

Q. You still wrote the attacks, though. True, and we say so. The magnitudes were fixed in advance — 15 to 25 failed passwords, 10 to 12 probed usernames — deliberately *above* the thresholds Wazuh's rules were written for, so we are not handicapping it. The next stage replaces the injected attacks with real output from real attack tools (hydra, nmap) run against a real SSH server.

Q. What is this "kill point" you mention? Before writing any of the measurement code, we wrote into the project's git history a rule: if the best robustness difference is 25 points or more, continue the project; if it is 10-25, do only the cheap parts; under 10, stop and publish the negative result. We measured 31.1. Writing it down first is what stops us moving the goalposts afterwards.

Q. Why five seeds? Because there are only 9 attack instances per corpus, so a single run moves the number in 33% steps. One run is not evidence. We pool five and recompute the rates from the summed counts rather than averaging percentages.

Q. Is your live demo a measurement? No, and the report says so. A live firing is one seed and three instances. The admissible numbers come from the five-seed recorded runs. The demo shows the mechanism; the report carries the evidence.

D. The hostile ones — practise these out loud

Q. Your precision is 0.29. That is terrible. It is, and it is our headline weakness. We raise 94 incidents where 9 attacks exist. The cause is structural: we score each line on its own, and one unusual line is genuinely weak evidence. We tested the obvious fix — making the model more conservative — and rejected it, because it improved precision on clean data while costing robustness under camouflage, which is the thing we are actually measuring. The real fix is to model *sequences* of events rather than single lines, and that is the next stage.

Q. Your own rule baseline beats your machine learning on this corpus. Yes — 0.96 F1 against our 0.43, and it is in our results file. That baseline is close to an oracle here: the test corpus is built from three attack templates and the baseline's rules were written knowing them. That is exactly why we moved to a third-party SIEM and real logs, where nobody wrote rules for our attacks.

Q. Where does your system lose to the SIEM? Three of the eight camouflage strategies. Rotating source IPs costs us 11.1 points and costs Wazuh nothing. Slowing the attack down costs us 8.9. The realistic combination — the one closest to a real attacker — costs us 13.3. One win, four ties, three losses. That is on the slide, not hidden in an appendix.

Q. Then why continue the project? Because the registered criterion was "25 points or more on at least one strategy", and rewording gives 31.1. The result is one mechanism rather than general robustness, and we describe it that way. And the losses are informative: they tell us the next thing to build is sequence modelling, which is exactly what would fix both `slow` and precision.

Q. Isn't 4,546 alerts from Wazuh just a configuration problem? Partly, and we checked rather than assumed. At its default alert level, a single successful login is already an alert — rule 5715, level 3. So it flags every successful login without correlating anything. We tried the higher alert levels too: at those it detects only 11% of attacks on the *clean* corpus, so its robustness there is a floor effect, not resistance. We report all three levels rather than picking the flattering one.

Q. You compressed time in some of those runs. Isn't that cheating? It would be, which is why those rows are labelled "illustration" and excluded from the result. Wazuh correlates on arrival time, so compressing time fixes its score by construction. The timing strategies are measured separately by a 6.5-hour replay at real speed — and that measurement is one of the ones we lose.

Q. This is just an anomaly detector with a chatbot bolted on. The detector is standard and we say so on the slide. What is not standard is the measurement: a pre-registered robustness criterion, a production SIEM as the baseline, eight camouflage strategies, five seeds, and published losses. We could not find a paper that does that for log analysis, and the eight journal papers attached to this presentation do not.

E. Practical and deployment

Q. What hardware does it need? Any laptop. No GPU, no cloud. A GPU makes the written explanation about four times faster — 3.9 seconds per incident instead of 18.2 — and nothing else changes.

Q. How fast is it? Would it keep up with a real server? Detection is a scoring pass over small batches — it keeps up with the log comfortably at demo volumes. The explanation is the slow part, which is why it is requested per incident by clicking rather than generated for all of them.

Q. Could it run on many machines? It already separates the part that collects logs from the part that analyses them, over HTTP, which is the first half of that. Our demo setup uses two laptops connected by one network cable: one runs the attacks and the SIEM, the other detects.

Q. Is any of it tested? 436 automated tests, run on every push through GitHub Actions. One of them pins the exact detection output — 751 lines, 130 flagged events, 94 incidents — so that if anyone changes detection accidentally, the test fails instead of the numbers quietly moving.

Q. Does it block attacks? No, deliberately. It recommends; it never acts. Automatic blocking on a detector with 0.29 precision would take down legitimate users several times a day.

F. What comes next

Q. What is the next piece of work? Sequence modelling — judging a *sequence* of events instead of each line on its own. It is the direct fix for precision and probably for the `slow` loss too, because a slow attack is still an unusual sequence even when each line looks ordinary.

Q. And after that? An entity graph so incidents can span users, hosts and IP addresses; the official MITRE data instead of our lookup table; automatic checking that every sentence the language model writes traces back to a real logged event; and attacks generated by real tools on a Kali machine instead of injected by us.

Q. What would you do differently if you started again? Build the measurement platform first. We built a detector first and then had to work out whether it was actually better than anything — the second version of this project deliberately reversed that order, and it is why we can answer any of these questions with a number.

G. Quick answers to keep in your pocket

If they ask	Say
Headline result	"Rewording costs Wazuh 33 points of detection and costs us 2. A 31-point difference, over five runs."
Weakest point	"Precision, 0.29. We raise 94 incidents for 9 real attacks. Sequence modelling is the fix."
Where you lose	"IP rotation, slowing down, and the realistic combination. Three losses out of eight."
Dataset	"Real OpenSSH logs from LogHub, 655,147 lines, plus a small synthetic corpus for regression testing."
Baseline	"The real Wazuh manager in Docker, with its own rules. We did not write them and cannot change them."
Novelty	"Not the detector. The measurement — pre-registered, against a production SIEM, with the losses published."